

# 안녕하십니까. **정확한 처리와 안정적인 서비스로** **사용자의 신뢰를 쌓는** 지원자 김도현입니다.

## Information

이름: 김도현

생년월일: 2000.03.28

이메일: kimdohyun032@gmail.com

전화번호: 010-7342-1227

GITHUB: [김도현 깃허브 링크](#)

## Table of Contents

|                      |       |           |
|----------------------|-------|-----------|
| Profile              | ..... | <u>3</u>  |
| Project – BE / Truve | ..... | <u>4</u>  |
| Project – AI / NaIQ  | ..... | <u>8</u>  |
| Product – FE / 시대생   | ..... | <u>10</u> |
| Project – FE / 시대팅   | ..... | <u>11</u> |
| Project – FE / 시간표   | ..... | <u>12</u> |



**협업을 즐기고, 항상 배우려 노력하는 개발자 김도현입니다.**  
**낮은 자세에서 타인의 의견을 존중하려 노력합니다.**

### Communication

**소통 능력을 인정 받아 팀을 끌여가는 자리에 나아갔습니다.**

- 교내 앱 개발 동아리 FE 챕터 리드 및 운영진
- 교내 캡스톤 디자인 팀장
- KT Cloud Tech UP 최종 프로젝트 개발 팀장 (기획, 디자인, 개발, 보안, 인프라, 총 15명 협업)

**다양한 사람들과 만나 교류하며 성장하고 있습니다.**

- 입학 후 교내 중앙 밴드 동아리 지속적 활동 [공연 링크](#)
- 교내 '드론활용 전문가 양성 교육' 참여, 항공 촬영 전문가 2급(민간인) 취득 [촬영 영상 링크](#)
- 동대문구 3개 대학 교류전 밴드 부문 우승
- 지구촌교회 분당채플 대학/청소년 지구 목자 섬김
- SSAFY 16기 서울캠퍼스 재학중

### Study

**웹 전반을 배우기 위해 노력하고 있습니다.**

#### FE

- React, TypeScript 관련 학습 및 앱/웹 개발
- '시대생' 앱 서비스 누적 다운로드 10,000건 기록
- '시대팅 시즌 6' DAU 1,000명, 결제 800건 달성

#### BE

- kt cloud TECH UP 백엔드 과정 1기 교육
- Java, Spring 기본 도구부터 Redis, Kafka 등 다양한 오픈소스 탐구 및 사용

# 김도현 - 프로필

## 학력

- 서울시립대학교 컴퓨터과학부(19.03 - 26.08)
- GPA 3.54/4.5 (전공 학점 3.64)

## 수상내역

- 서울시립대학교 캡스톤디자인 우수작품상 수상 (25.01.05)

## 자격 및 어학

- 정보처리기사
- SQLD
- AWS SAA
- ADsP
- TOEIC Speaking IH(150)

## 대외 활동

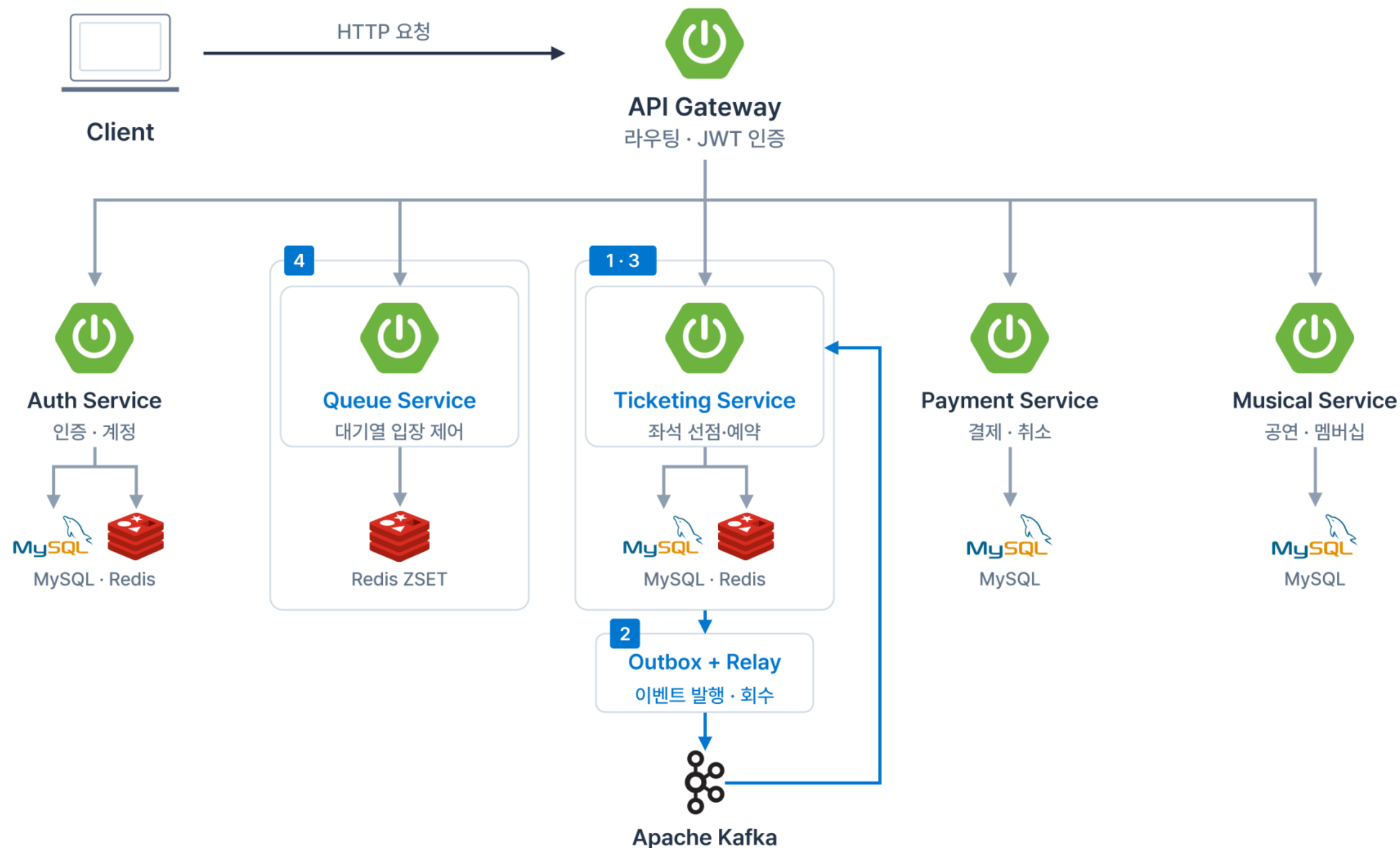
1. 교내 앱 개발 중앙동아리 **UOSLIFE** (24.03 - 현재)
  - 25-1학기 FE 챗터 리드
  - '시대생' 앱 운영
2. 반찬가게 픽업 플랫폼 **맘찬픽** (24.08 - 25.09)
  - 팀장 및 FE 개발 담당
  - 서울시립대학교 캡스톤 디자인 우수작품상 수상
3. 42Seoul 9기 본과정 (23.03 - 23.11)
4. 교내 밴드 중앙동아리 THRASH AL (19.03 - 현재)

## 추가 교육

- 삼성청년SW·AI아카데미(SSAFY) (26.07 - 현재)
- kt cloud TECH UP 백엔드 과정 (25.09 - 26.04)

# 1. Truve kt cloud TECH UP (26.01 – 26.04, 26.08 – 진행중) [GITHUB 링크](#)

뮤지컬 팬덤을 위한 예매 플랫폼 | Java · Spring Boot · Redis · MySQL · Kafka



## 개인 핵심 기여 사항

- 1 좌석 선점과 실패 처리
  - 예약·이벤트 발행 단일 트랜잭션으로 저장
  - 실패 시 소유권 확인 이후 선점 해제
- 2 이벤트 발행과 장애 복구
  - 이벤트 선점과 Kafka 전송 분리
  - 중단 작업 회수 이후 재발행 시도
- 3 JVM 워업으로 초기 응답속도 개선
  - 초기 응답 속도 약 83% 개선
- 4 대기열 기반 진입 제어
  - Redis ZSET 기반 대기열 구성
  - 순번별 폴링 주기 제어

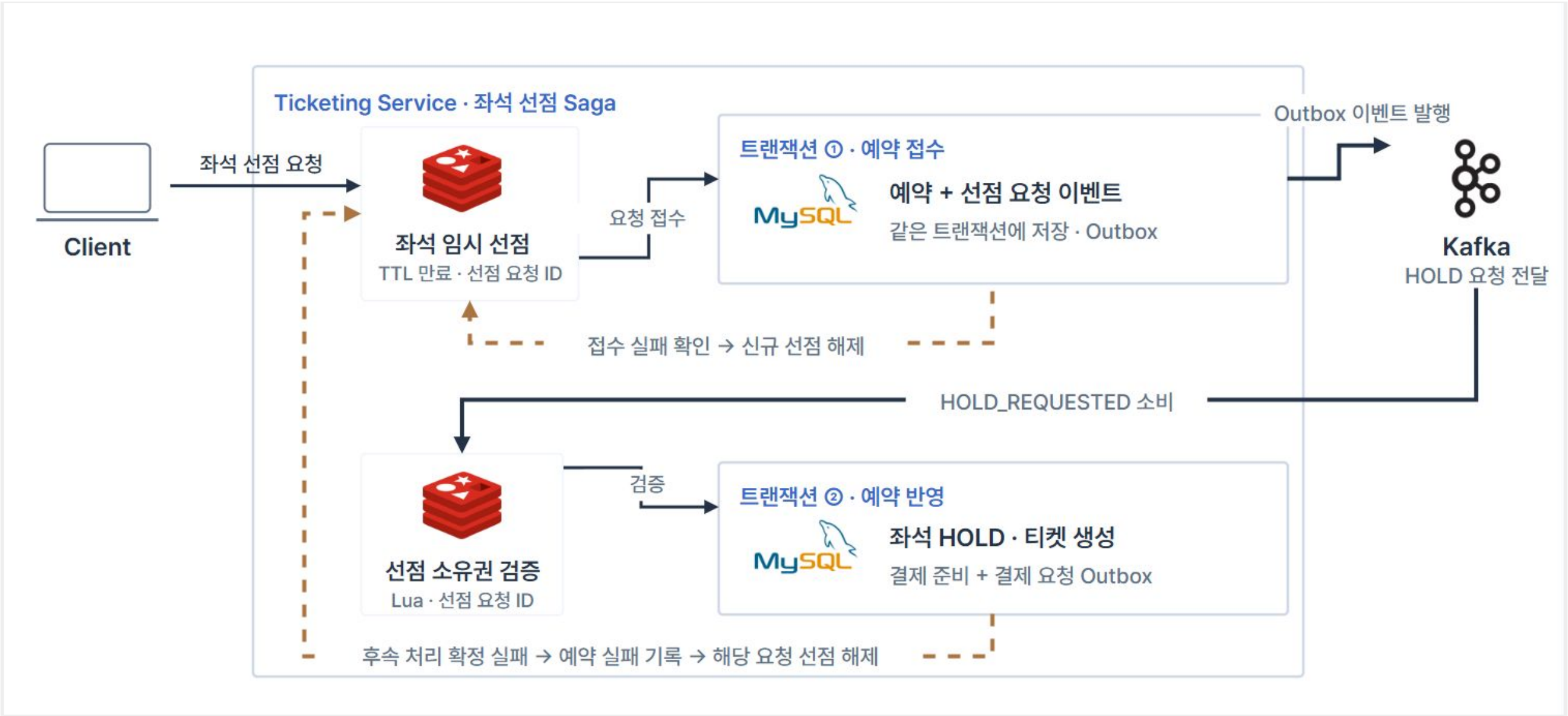
# Truve 기여 사항 ① - Saga 기반 좌석 선점과 보상

Redis 좌석 선점과 DB 예약 저장을 단계별로 분리하고, 실패가 확인된 요청의 선점만 해제하도록 설계했습니다.

Redis 좌석 선점 후 DB 예약 저장이 실패하면, 예약 없이 좌석만 점유될 수 있습니다.  
반대로 DB 커밋 여부가 불확실한 상황에서 선점을 해제하면, 이미 저장된 예약의 좌석이 다른 요청에 다시 선점될 수 있습니다.

Redis 좌석 선점 → DB 예약 저장 시도 → 저장 실패 확정 → Lua로 선점 소유권 검증 → 해당 선점 해제

오케스트레이션 기반 Saga로 처리 단계를 조율하고, 실패 확정 시 소유권을 검증해 보상 트랜잭션 수행



## ① 예외 발생 후 DB 저장 여부 확인

DB를 재조회해 실제 저장 여부를 확인,  
저장 실패가 확인된 경우에만 보상 트랜잭션 실행

## ② 요청 소유권을 검증한 조건부 해제

Lua로 선점 요청 ID를 대조,  
다른 요청이 획득한 좌석 해제 방지

## ③ 예약 실패 후 재시도 가능한 상태로 복구

후속 처리 실패 확정 시 예약 실패 기록·해당 선점 해제  
좌석 점유가 남아 재예매를 막는 상황 방지

# Truve 기여 사항 ② - Outbox Relay의 이벤트 발행과 장애 복구

이벤트 선점과 Kafka 전송을 분리하고, 중단된 발행 작업은 다른 Relay가 이어받도록 설계했습니다.

Kafka 응답 대기까지 DB 트랜잭션에 포함하면, 네트워크 지연 동안 DB 락이 유지됩니다.  
작업을 선점한 Relay가 중단되면, 완료되지 않은 이벤트를 회수해 다시 발행해야 합니다.

이벤트 선점·커밋 → DB 락 해제 → Kafka 발행·ACK → 작업 소유권 검증 → 완료 기록

Transactional Outbox에 저장된 이벤트를 Relay가 발행하고, 점유 시간이 만료된 작업은 재선점해 복구



## ① Kafka 응답 대기과 DB 락 분리

이벤트 선점 후 커밋, DB 트랜잭션 밖에서 Kafka 발행  
네트워크 응답 대기로 인한 DB 락 장기 점유 방지

## ② 작업 소유권을 검증한 조건부 완료

재선점 시 작업 소유권 토큰 갱신, 일치할 때만 완료 기록  
이전 Relay의 뒤늦은 완료 갱신 방지

## ③ 중단된 발행 작업 회수·재발행

작업 점유 만료 후 다른 Relay가 미완료 이벤트 재선점  
Relay 중단으로 멈춘 이벤트 발행 재개

재발행 시 중복 전송 가능성을 고려해, 소비 측 멍등 처리와 함께 설계했습니다.

# Truve 기여 사항 ③ - JVM 워업으로 초기 요청 지연 완화

서버 기동 직후의 실행 경로를 미리 준비하고, API 반복 호출로 JIT 컴파일을 유도해 초기 응답 지연을 줄였습니다.

기동 완료만으로 실제 요청 경로의 준비가 충분하지 않았고, 새 Pod 기동 시 응답시간이 지연되었습니다.  
이에 워업을 도입해 실제 API 호출을 진행했습니다. 반복 횟수에 따른 컴파일 기록·첫 응답·준비 시간을 비교했습니다.

## Warm-up 도입



## JVM 워업 효과

localhost 환경에서 실제 API 호출

57.04 ms → 9.48 ms

서버 기동 직후 응답 지연 83.4% 감소

### 컴파일·응답·준비 비용 비교

| HTTP 호출   | 0회     | 100회   | 500회   | 2,000회 |
|-----------|--------|--------|--------|--------|
| C1 (건)    | 10,511 | 12,184 | 13,810 | 14,829 |
| C2 (건)    | 1,335  | 1,532  | 1,850  | 2,379  |
| 첫 응답 (ms) | 57.04  | 11.06  | 9.48   | 9.56   |
| 준비 (초)    | —      | 1.32   | 5.40   | 16.05  |

C1·C2: 측정 요청 시작 전 JVM 전체 컴파일 결과 생성 건수(nmethod)

## ① HTTP 경로까지 워업 확장

로컬에서 실제 API를 호출해  
필터·MVC·Heartbeat 경로 사전 실행

## ② 반복 실행으로 JIT 최적화 유도

C2 최적화를 고려해 반복 횟수를 늘려 비교  
C1·C2 컴파일량과 첫 응답 지연을 함께 확인

## ③ 준비 비용을 고려해 500회 선택

2,000회는 C2가 늘어도 첫 응답 지연 개선 효과 미미  
준비 시간은 16.05초로 크게 증가

## 2. NalQ

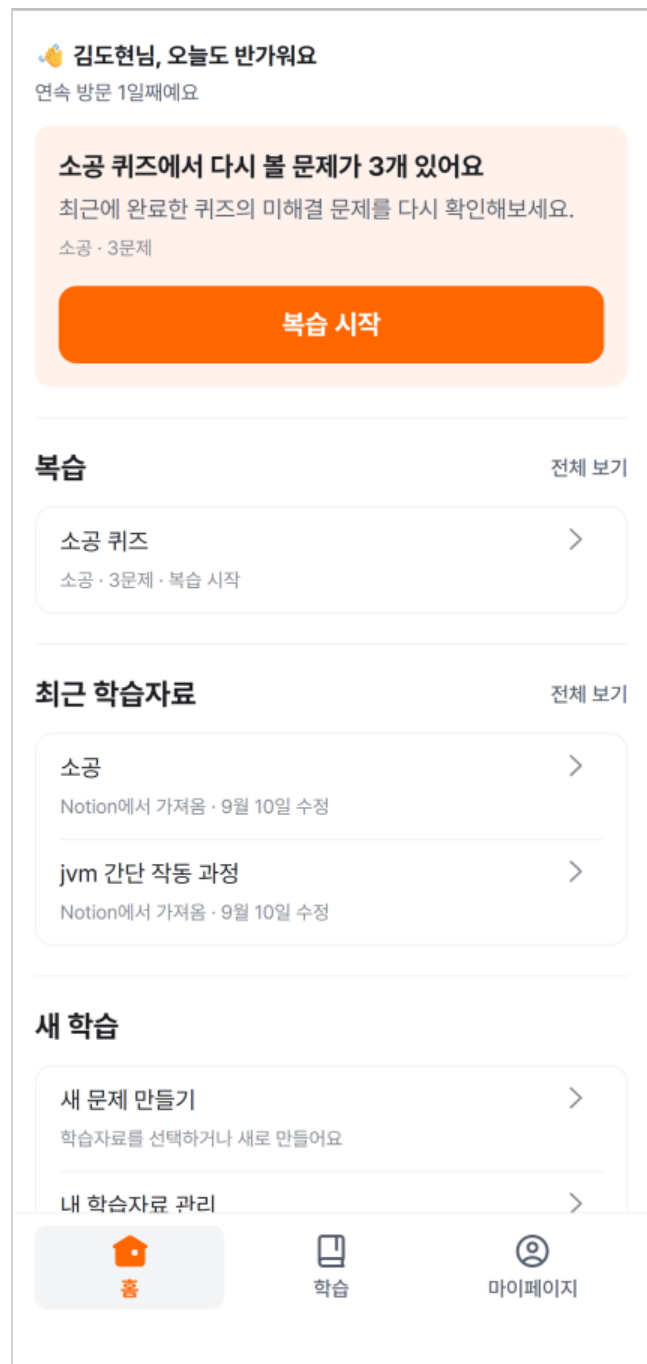
AI 활용 개인 프로젝트 (26.08 – 진행중)

[nalq.app 링크](https://nalq.app)

[GITHUB 링크](#)

내 학습자료를 퀴즈와 복습으로 연결하는 플랫폼 | Java · Spring Boot · MySQL · Redis · React · AWS

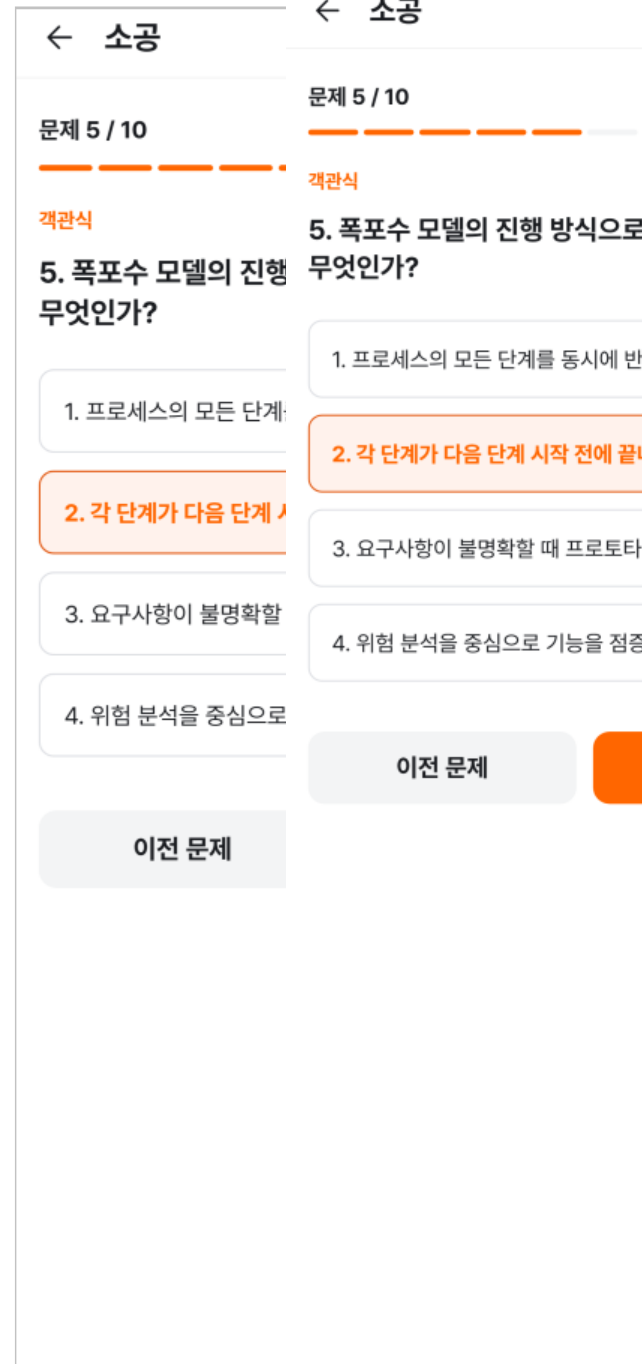
AI가 구현할 맥락과 작업 범위를 설정하고, 테스트와 변경 검토로 결과를 확인했습니다.



▲ 메인 화면



▲ 문제 생성 화면



▲ 문제 풀이 화면



▲ 채점 결과 화면

## 개인 핵심 기여 사항

### 1 AI가 참고할 맥락과 판단 기준 설정

- 기능 목적·사용자 흐름·예외 선정
- 확정 사항과 열린 질문 구분
- PRD·UX·Contract·TRD 문서 작성

### 2 에이전트의 역할과 수정 범위

- 작업 별 전문 에이전트 세팅
- 검토 가능한 작업 단위 위주로 범위 지정
- AGENTS.md를 통한 지침 제어

### 3 구현 결과를 확인할 검증 기준

- 로컬과 CI 동일 검증 기준 사용
- 생성 규칙과 실패 조건을 테스트로 확인

### 4 디자인 일관성 확보

- 당근 SEED 디자인 시스템 오픈 소스 활용
- DESIGN.md와 결합해 일관된 UX/UI

AI를 활용한 개발 방식으로 2주 내 웹 MVP를 배포했습니다.

기획·설계·개발·배포·운영까지 웹 서비스 전 생애주기를 경험하고 있으며, 현재 모바일 앱 배포를 준비하고 있습니다.

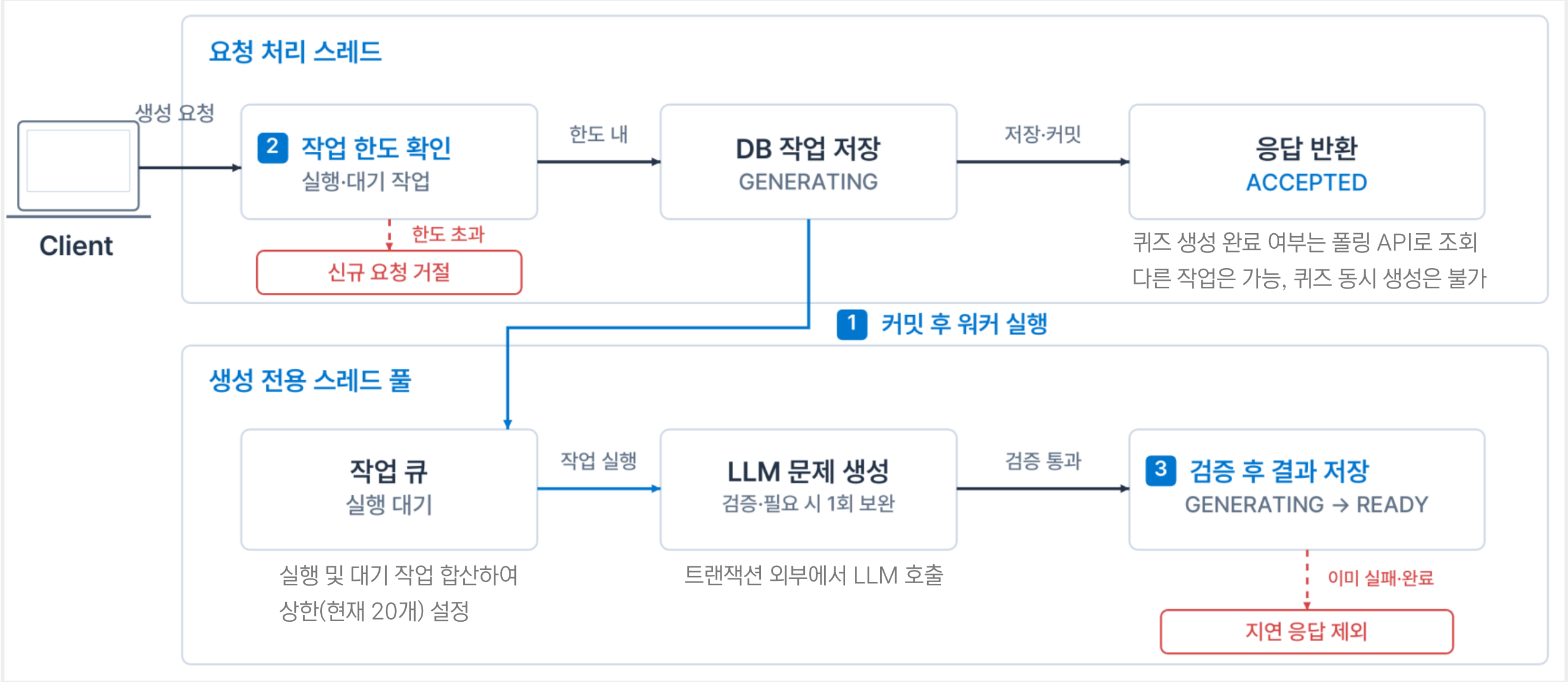
# NalQ 핵심 설계 - LLM 문제 생성 비동기 처리와 상태 관리

LLM 문제 생성을 전용 워커로 분리하고, 동시 작업 수와 결과 저장 조건을 제어했습니다.

LLM 호출을 동기 처리하면 응답을 기다리는 동안 요청 처리 스레드가 점유됩니다.  
비동기 처리에서도 작업이 과도하게 쌓이거나, 실패한 작업이 늦은 응답으로 완료처리 될 수 있습니다.

작업 한도 확인 → DB 작업 저장 → 워커 실행 → 문항 및 작업 상태 검증 → 결과 저장

요청 처리와 문제 생성을 분리하고, 검증된 결과만 저장 트랜잭션에서 확정되어 문제가 생성됩니다.



## ① 요청 처리와 생성 작업 분리

DB 커밋 후 전용 워커에서 생성 실행  
생성 완료 대기 없이 요청에 응답

## ② 동시 문제 생성 작업 수 제한

실행·대기 작업의 수를 합산해 상한 설정  
한도 초과 시 신규 시도 유저 재시도 안내 메시지 표시

## ③ 작업 상태 검증 후 저장

문항 검증 후 DB 행 잠금·상태 확인  
GENERATING인 작업에만 결과 반영

LLM의 문제 생성을 기다리지 않는 요청 흐름을 구성하고, 작업의 과도한 누적과 성공/실패 후 퀴즈 데이터 일관성을 확보했습니다.

### 3. 시대생

UOSLIFE (24.03 – 25.08)

UOSLIFE 팀페이지

Google play store

App store

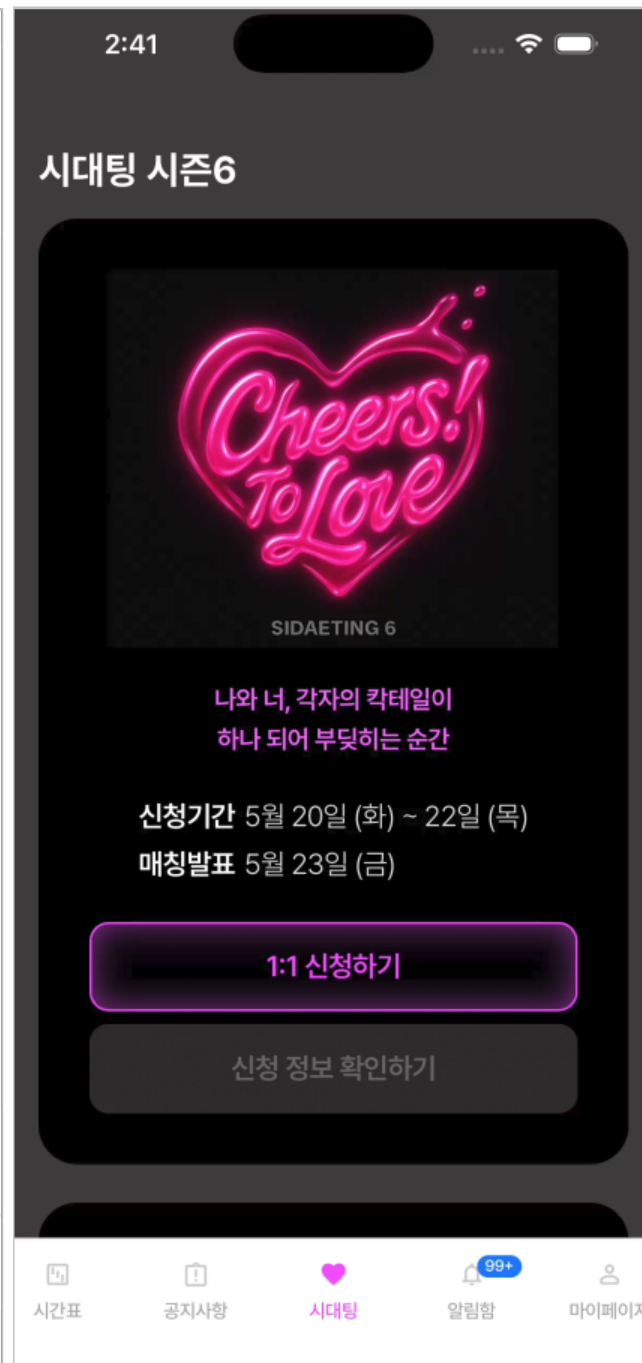
서울시립대학교 교내 학사 정보 제공 앱 | TypeScript · React · React Native · TanStack Query · Jotai

시대생 FE 개발 챕터로 참여해 재학생이 실제 사용하는 서비스를 개발하고, 출시 후 운영과 개선을 이어갔습니다.

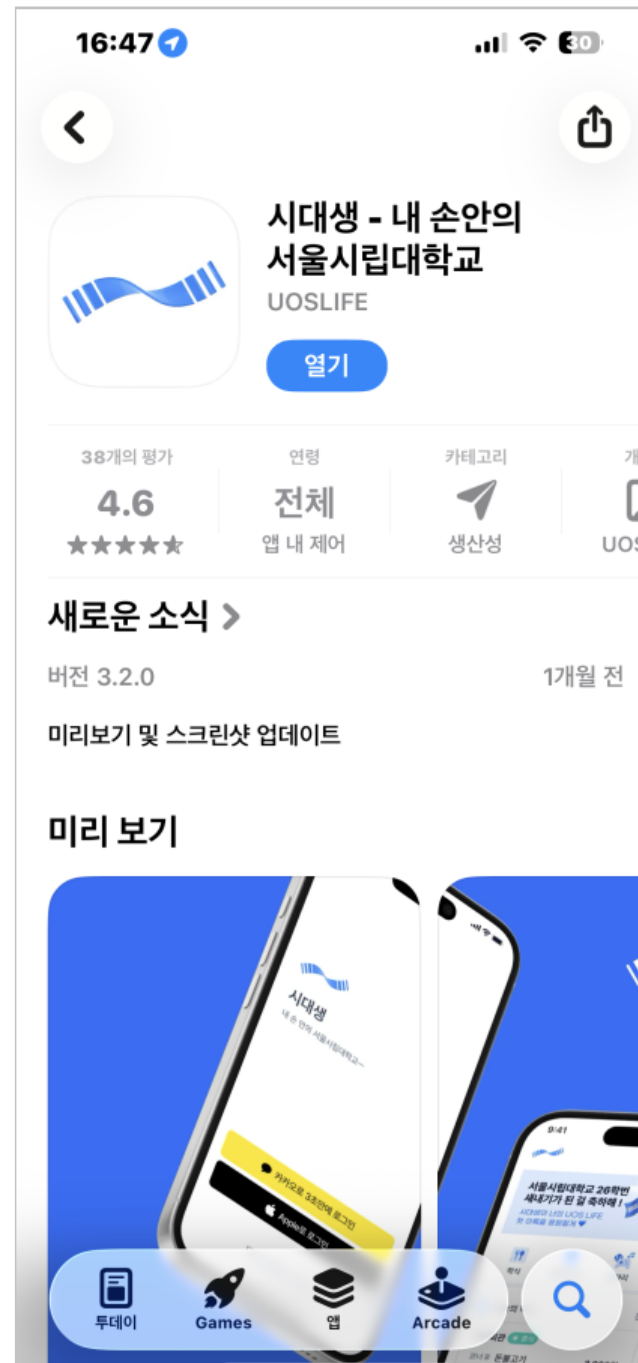
사용자 피드백을 기능에 반영하며, 학생들의 일상 속 불편을 해결하는 개발에 보람을 느꼈습니다.



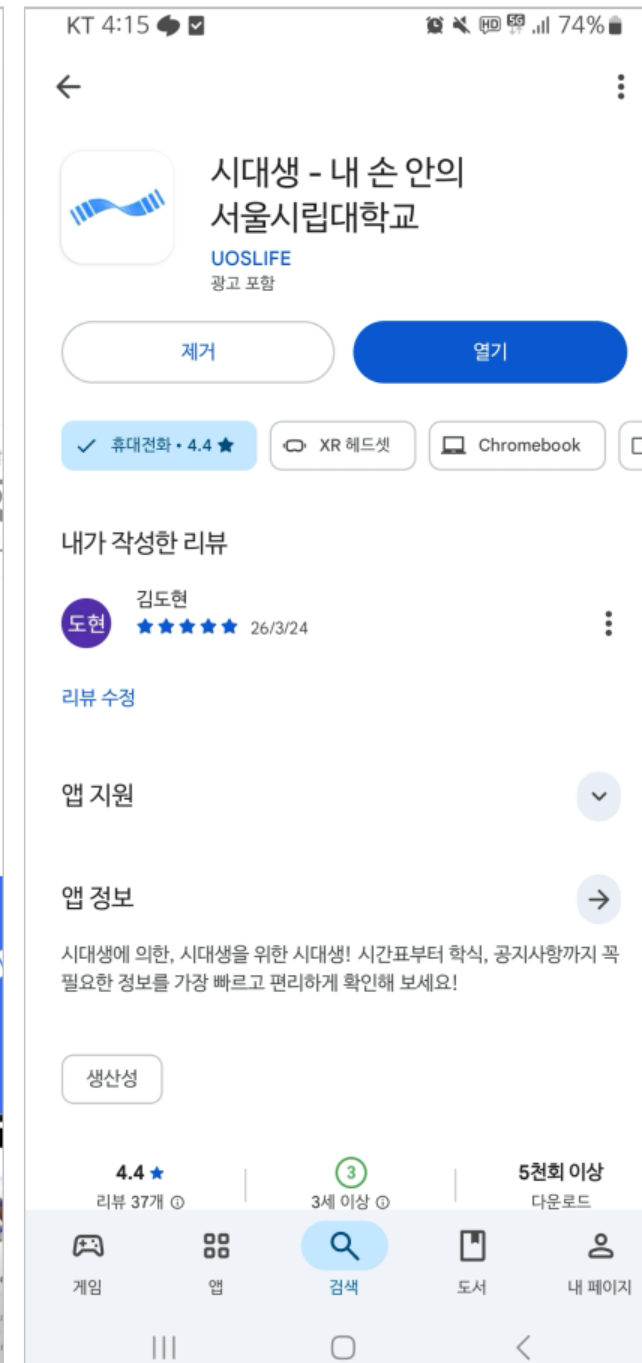
▲ 시간표 화면



▲ 시대팅 입장 화면



▲ 앱스토어 배포



▲ 구글 플레이 스토어 배포

### 프로덕트 내 핵심 기여 사항

#### 1 시대팅 시즌6 FE 챕터

- 다단계 신청·퍼널 입력 상태 관리
- 앱 내 PortOne 결제 연동
- 저대역폭 결제 흐름 검증

#### 2 시간표 기획 및 설계

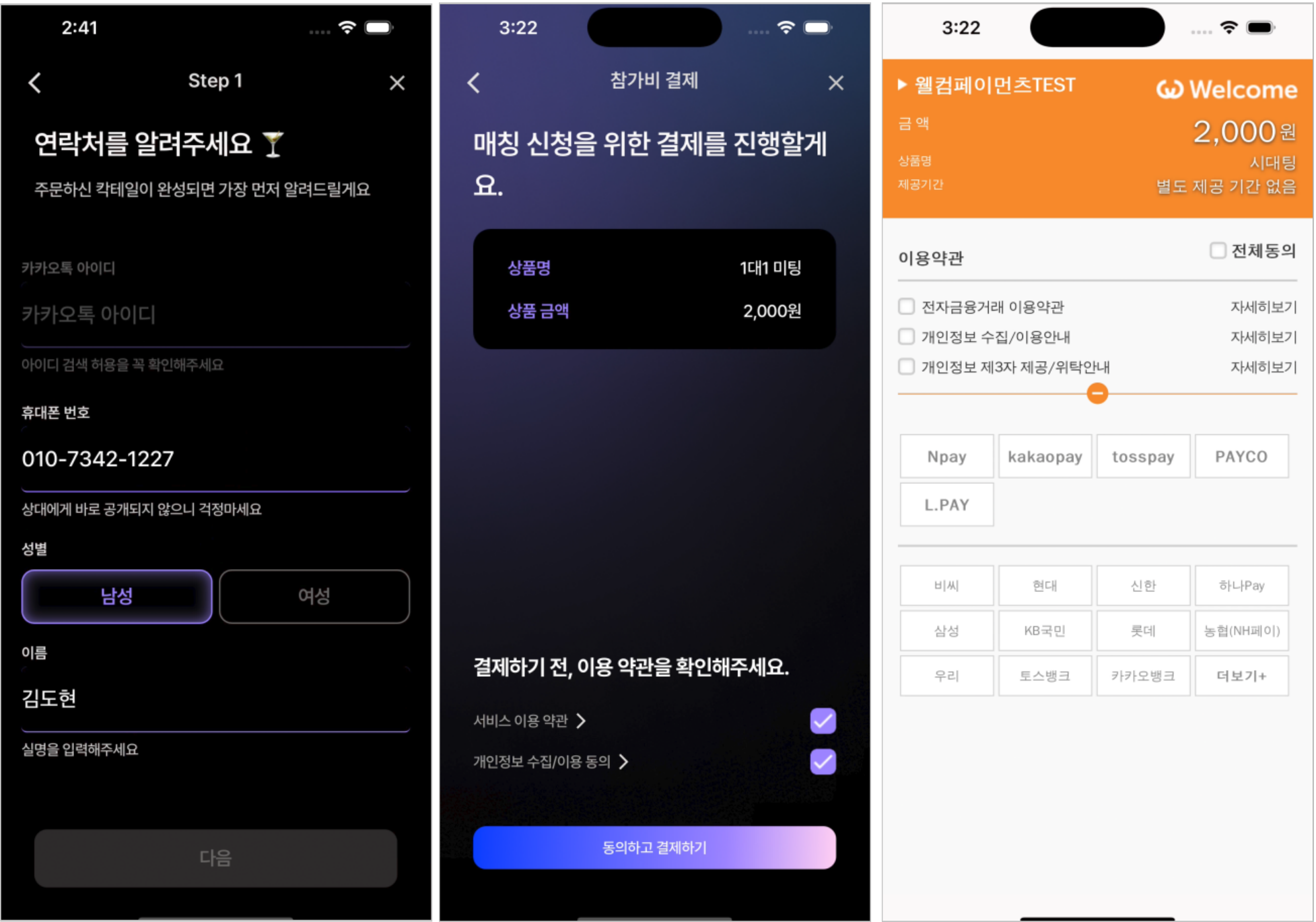
- 시간표 UI·상호작용 구현
- 친구 관리·카카오톡 공유
- 플랫폼별 딥링크 전달 방식 개선

시대생을 통해 **누적 다운로드 1만 건 이상**의 앱을 꾸준히 유지보수·개선하며, 실사용자 서비스의 운영 경험을 쌓았습니다.

# 시대생 기여 사항 ① – 시대팅 시즌6 신청 및 결제 흐름 구현

UOSLIFE · '시대생' 앱 내 서비스 '시대팅 시즌6' / 2025.03 – 2025.06

인적사항 · 내 정보 · 이상형 정보로 이어지는 신청 흐름과 앱 내 결제 모듈을 연동했습니다.



▲ 시대팅 정보 입력 화면

▲ 시대팅 결제 입장 화면

▲ PortOne 결제 모듈

## 개인 핵심 기여 사항

### ① 퍼널 · 타입으로 입력 상태 관리

- useFunnel을 활용해 입력 단계를 나누고 단계별 타입 정의
- 기존 입력 저장 상태에 따라 미완료 단계로 진입 유도
- 입력 중 이탈해도 미완료 단계부터 이어 작성하도록 개선 – 이전 시즌 피드백

### ② 앱·웹 간 결제 모듈 연동

- React Native 앱 내 PortOne 결제 모듈 연동, 브릿지로 웹뷰 내 전달
- 백엔드 팀과 결제 처리 및 웹훅 연동 점검

### ③ 저대역폭 환경에서 결제 검증

- 축제 기간 내 교내 네트워크 품질 저하 예상
- Network Link Conditioner로 1KB/s 수준 저대역폭 환경 결제 흐름 검증

프로젝트 성과 | 유료 결제 800건 이상 · 결제 CS 10건 이하

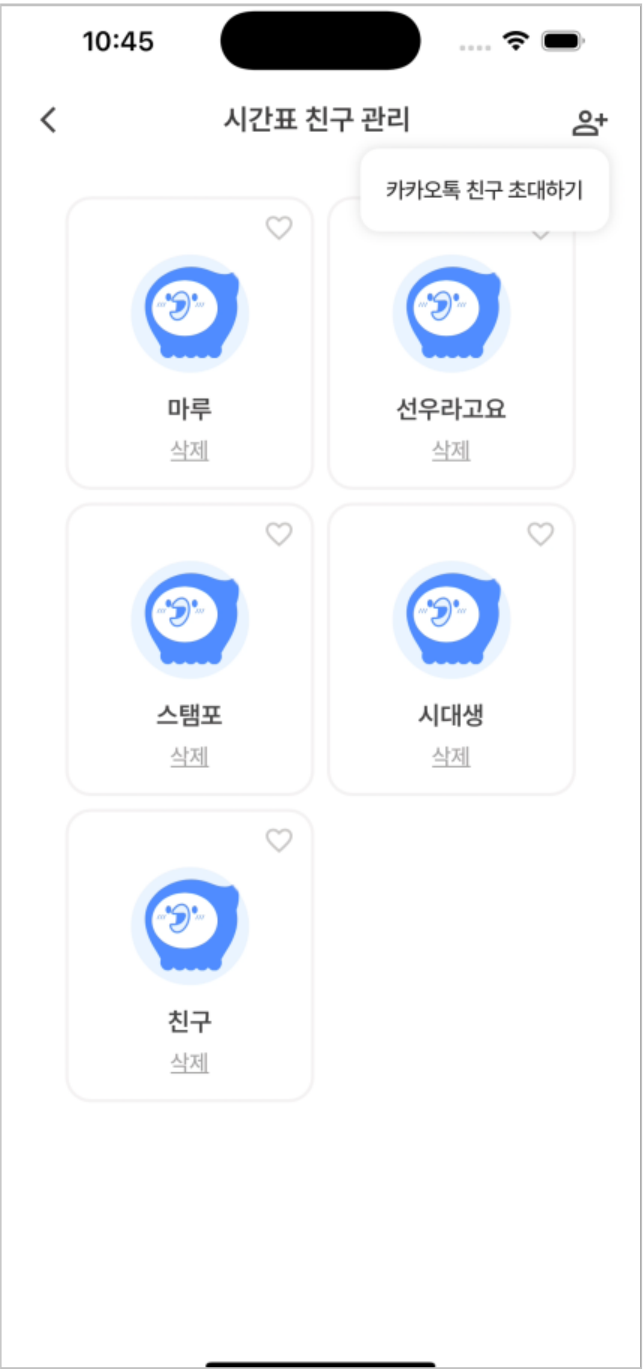
# 시대생 기여 사항 ② - 시간표 기능 및 공유 흐름 개선

UOSLIFE · '시대생' 앱 내 서비스 '시간표' / 2024.12 - 2025.08

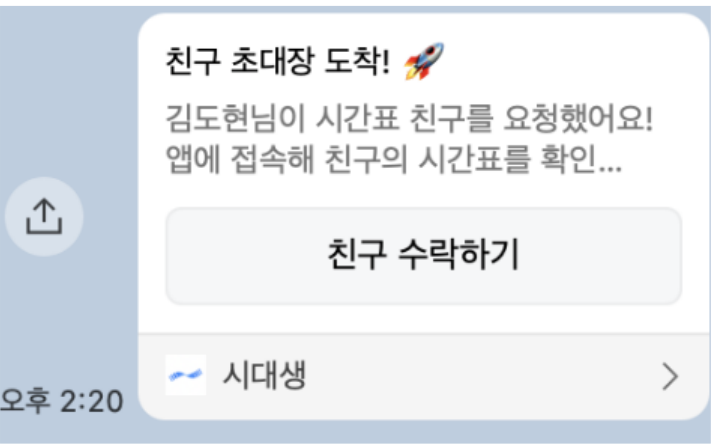
시간표 편집 기능과 친구 시간표 조회·공유 흐름을 구현했습니다.



▲ 시간표 메인 화면



▲ 친구 시간표 조회 화면



▲ 카카오톡 초대 메시지

## 개인 핵심 기여 사항

### ① UI와 비즈니스 로직 분리

- 시간표 테이블·과목·카드·오버레이 구현
- 컴포넌트의 복잡한 상호작용을 Hook으로 분리하여 유지보수성 개선

### ② 친구 시간표 및 카카오톡 공유 구현

- 시간표 친구 관리·좋아요·공유 기능 구현
- 초대 링크에 친구 정보를 담아 딥링크로 연결

### ③ 플랫폼별 이벤트 메시지 처리 방식 핸들링

- 안드로이드는 초대 링크 접속 시 메인 화면으로 접속되는 문제 발생
- 앱·웹 간 통신 문제로 좁히고, 안드로이드/iOS 간 메시지 처리 방식 파악
- OS별 메시지 수신 객체 분리, 웹뷰 로드 직후 브릿지 초기화 이벤트 전송
- 시대팅·공지사항 등 타 서비스에서도 활용할 수 있는 딥링크 기반 마련

프로젝트 성과 | 친구 시간표 조회·공유 기능 배포 이후 WAU 30% 증가

**다우기술에서도 고객님의 자산을 정확히 처리하고  
안정적인 서비스를 제공할 것입니다.  
지금까지 지원자 김도현이었습니다.**

|                      |       |           |
|----------------------|-------|-----------|
| Profile              | ..... | <u>3</u>  |
| Project – BE / Truve | ..... | <u>4</u>  |
| Project – AI / NaIQ  | ..... | <u>8</u>  |
| Product – FE / 시대생   | ..... | <u>10</u> |
| Project – FE / 시대팅   | ..... | <u>11</u> |
| Project – FE / 시간표   | ..... | <u>12</u> |